Matrix Free Solver

High-Performance Solver for the ADR Equation

Cristiano Corona Daniele Confalonieri Dmitrii Meshcheriakov Giorgio Mantoan

Politecnico di Milano
High Performance Computing Engineering, NMPDE

Academic Year 2025-2026

Outline

1. Mathematical Problem & Weak Formulation
2. Preconditioning & Dirichlet Conditions
3. Software Verification
4. Hardware Profiling & Scalability

Overcoming the Memory Wall

Matrix-Based Limitations:

- Explicit storage of A is inherently memory-bound.
- Local dense matrices scale as $\mathcal{O}(p^{2d})$.

deal.II Implementation:

- `FEEvaluation` replaces `FEValues`, processing batches of cells using AVX-512 SIMD instructions.
- A custom `ADROperator` isolates the PDE physics.
- Spatial coefficients (μ, $\boldsymbol{\beta}$, γ) are pre-computed and packed into `VectorizedArray` structures for L1 cache alignment.

Matrix-Free via Sum-Factorization:

- Decouples multidimensional sums into a sequence of nested univariate contractions.
- Drops operations count from $\mathcal{O}((p+1)^{2d})$ to $\mathcal{O}(d \cdot (p+1)^{d+1})$.
- Computes the action of the operator "on-the-fly".

The Advection-Diffusion-Reaction Problem

Strong Form:

$$-\nabla \cdot (\mu \nabla u) + \nabla \cdot (\boldsymbol{\beta} u) + \gamma u = f \quad \text{in } \Omega$$

Weak Formulation (Standard):

$$\int_\Omega \mu \nabla u \cdot \nabla v \, d\boldsymbol{x} + \int_\Omega \nabla \cdot (\boldsymbol{\beta} u) v \, d\boldsymbol{x} + \int_\Omega \gamma_{eff} \, uv \, d\boldsymbol{x} = \int_\Omega fv \, d\boldsymbol{x} + \int_{\Gamma_N} hv \, d\sigma$$

Matrix-Free Optimization (γ_{eff}):

- Using the product rule: $\nabla \cdot (\boldsymbol{\beta} u) = \boldsymbol{\beta} \cdot \nabla u + u(\nabla \cdot \boldsymbol{\beta})$.
- Grouping zeroth-order operators into an effective reaction term:

$$\int_\Omega \mu \nabla u \cdot \nabla v \, d\boldsymbol{x} + \int_\Omega v(\boldsymbol{\beta} \cdot \nabla u) \, d\boldsymbol{x} + \int_\Omega (\gamma + \nabla \cdot \boldsymbol{\beta}) uv \, d\boldsymbol{x} = \dots$$

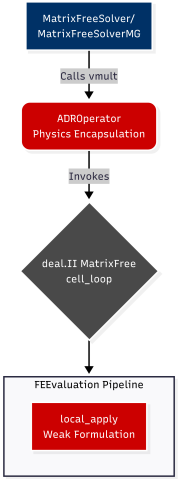
Software Architecture: The Matrix-Free

1. Separation of Concerns

- `MatrixFreeSolver`: Handles MPI, Krylov solvers, and Multigrid logic.
- `ADROperator`: Encapsulates the PDE physics, exposing a generic linear operator interface.

2. The Computational Core

- `local_apply`: Evaluates weak form on-the-fly.
- Uses `FEEvaluation` instead of `FEValues`.
- `cell_loop`: Dynamically batches elements for SIMD (AVX-512).
- `VectorizedArray`: Pre-computed coefficients for L1 cache alignment.



The Sum-Factorization Algorithm

Algorithm ($d = 3$ Case):

- Direct evaluation of u for all $(p+1)^d$ quadrature points would require $\mathcal{O}((p+1)^{2d})$ operations.
- Sum-Factorization decouples the evaluation into three 1D steps:

$$1.\ v_{i,j,s} = \sum_{k=0}^{p} u_{i,j,k}\varphi_k(\zeta_s) \quad \text{(Contraction along } \zeta\text{)}$$

$$2.\ w_{i,r,s} = \sum_{j=0}^{p} v_{i,j,s}\varphi_j(\eta_r) \quad \text{(Contraction along } \eta\text{)}$$

$$3.\ u(\xi_q, \eta_r, \zeta_s) = \sum_{i=0}^{p} w_{i,r,s}\varphi_i(\xi_q) \quad \text{(Contraction along } \xi\text{)}$$

- Each step corresponds to a dense matrix-matrix multiplication, easily vectorized using SIMD instructions (e.g., AVX-512) on modern CPUs. The cost becomes $\mathcal{O}(d(p+1)^{d+1})$.

Preconditioning & Boundary Handling

Geometric Multigrid (GMG):

- Ideal for Matrix-Free since transfers rely on matrix-vector products.
- **Symmetric systems:** Chebyshev iterations smooth high-frequency errors associated with Jacobi-preconditioned eigenvalues.
- **Non-symmetric systems:** Dynamic selection falls back to Damped Jacobi and GMRES outer solver.

Inhomogeneous Dirichlet (Γ_D):

- The iterative solver expects a linear operator implicitly solving the homogeneous problem.
- **Hybrid Explicit Integration:** The exact boundary shift is evaluated explicitly via `FEValues` during RHS assembly.

Method of Manufactured Solutions (MMS)

- Matrix-free operators exist only as functions; correctness must be verified by analyzing solution convergence against a manufactured analytical solution u_{ex}.

Problem 1: Homogeneous Dirichlet

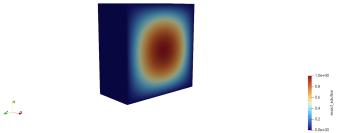
- ADR equation with constant fields: $\mu = 1, \boldsymbol{\beta} = (1, 1, 1)^T, \gamma = 1$.
- Exact solution: $u_{ex}(x, y, z) = \sin(\pi x)\sin(\pi y)\sin(\pi z)$.

Problem 2: Inhomogeneous Dirichlet

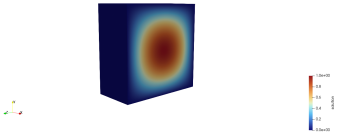
- Symmetric Diffusion-Reaction problem: $\mu = 1, \boldsymbol{\beta} = (0, 0, 0)^T, \gamma = 1$.
- Exact solution: $u_{ex}(x, y, z) = e^{x+y+z}$.

Verification 1: Homogeneous Dirichlet (Qualitative)

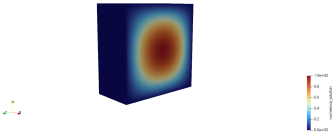
Visual Inspection ($p = 3$, 5 Global Refinements)



Exact Solution

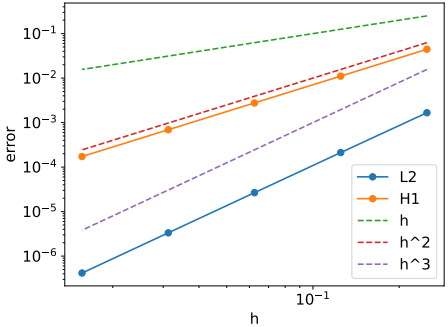


Matrix-Based

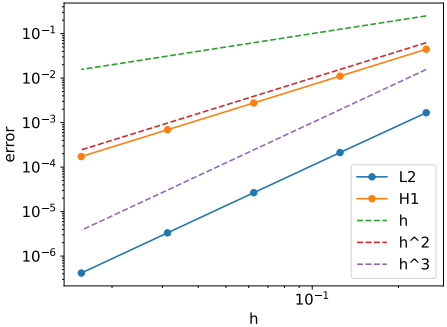


Matrix-Free

Verification 1: Error Convergence Analysis ($p = 2$)



Matrix-Based (L^2 & H^1 errors)

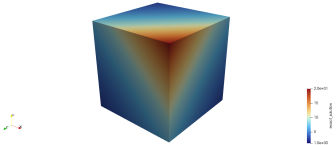


Matrix-Free (L^2 & H^1 errors)

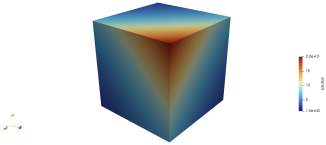
Both solvers match theoretical asymptotes ($\mathcal{O}(h^3)$ for L^2, $\mathcal{O}(h^2)$ for H^1).
On-the-fly sum-factorization introduces no additional integration errors.

Verification 2: Inhomogeneous Dirichlet (Qualitative)

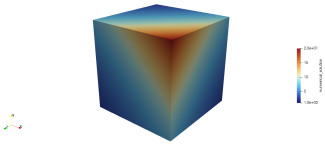
Validating Hybrid Explicit Integration ($p = 3$)



Exact Solution

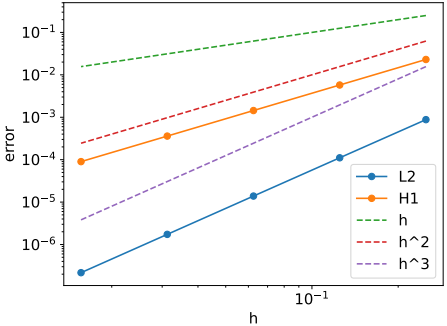


Matrix-Based

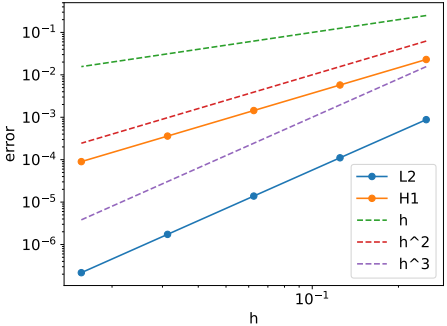


Matrix-Free

Verification 2: Error Convergence Analysis ($p = 2$)



Matrix-Based (L^2 & H^1 errors)



Matrix-Free (L^2 & H^1 errors)

Profiling: Objectives & Environments

- **Objective:** Comprehensively compare the computational efficiency of the Matrix-Free solver against the standard Matrix-Based implementation.

Testing Environments:

- **HPC Polimi Cluster:** Used for scalability testing and throughput analysis.
- **Local Workstation:** Dedicated to low-level architectural profiling (e.g., precise RAM usage and CPU cache behavior).

Profiling Metrics & Variable Parameters

Metrics Evaluated:

- **Memory Usage:** Peak footprint reduction.
- **Cache Efficiency:** Cache hit/miss percentages across L1 and LLC.
- **Scalability:** Both Strong and Weak scaling behaviors across multiple nodes.
- **Throughput:** Computational speed measured in Degrees of Freedom (DoFs) per second.

Parameters Varied:

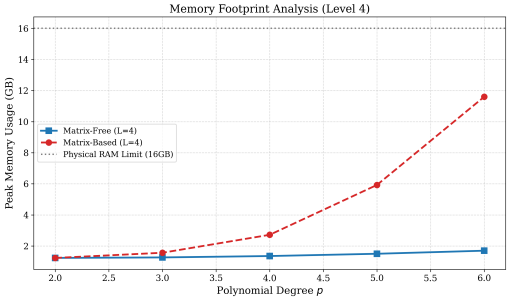
- **Mesh Refinement (l):** Altering the spatial grid resolution to increase total DoFs.
- **Polynomial Degree (p):** Testing low to high-order finite element approximations.
- **Hardware Concurrency:** Scaling the number of active MPI processes/cores.

Local Workstation Specifications

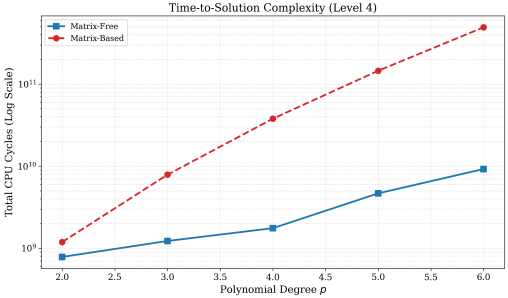
Local Profiling Workstation Specifications	
Processor Model	AMD Ryzen 7 8845HS
Architecture	x86_64 (Zen 4)
Cores / Threads	8 Physical Cores / 16 Logical Threads
Max Boost Clock	Up to 5.1 GHz
L1 Cache	256 KiB Data + 256 KiB Instruction (8 instances)
L2 Cache	8 MiB (8 instances)
L3 Cache (LLC)	16 MiB (Shared)
SIMD Extension	AVX-512
Physical Memory	16 GB

Table: Hardware specifications of the local machine used for low-level architectural profiling.

Hardware Profiling: Memory & CPU Limits



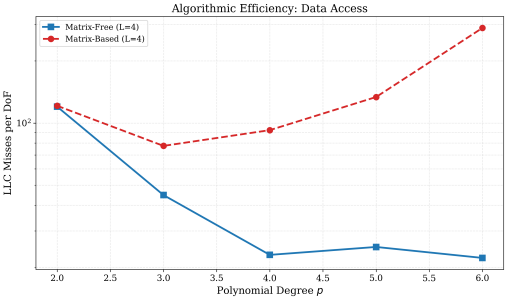
Peak Memory Usage



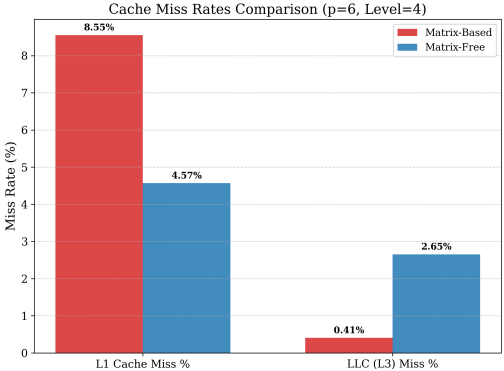
Total CPU Cycles (Log Scale)

- Matrix-Based approaches RAM limits at $p = 6$ (~ 12 GB), while Matrix-Free stays virtually flat.
- Matrix-Free reduces required CPU cycles by orders of magnitude.

Arithmetic Intensity & Cache Utilization



LLC Misses per DoF



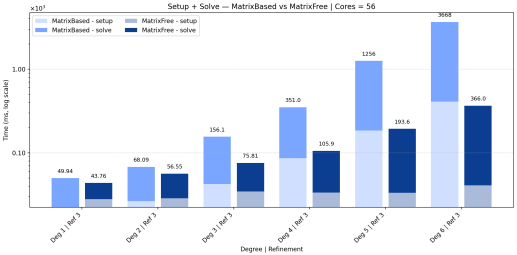
Cache Miss Rates ($p = 6$)

- Decreasing LLC misses per DoF proves high arithmetic intensity; operations are performed on data already in the L1 cache.
- L1 miss rate drops from 8.55% to 4.57% due to vectorized loop locality.

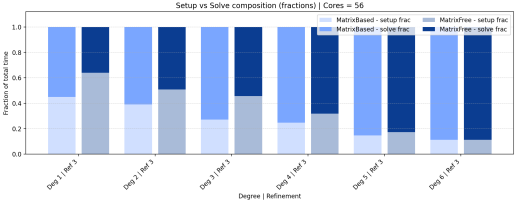
HPC Cluster Specifications

HPC Cluster Hardware Specifications	
Processor Model	Intel Xeon Gold 6238R @ 2.20 GHz
Architecture	x86_64 (Cascade Lake)
Sockets / Cores / Threads	2 Sockets / 56 Cores / 112 Threads
Base Frequency	2.20 GHz
L1 Cache	1.8 MiB Data + 1.8 MiB Instruction (56 instances)
L2 Cache	56 MiB (56 instances)
L3 Cache (LLC)	77 MiB (2 Shared)
SIMD Extension	AVX, AVX2, AVX-512 (F, DQ, CD, BW, VL, VNNI)
NUMA Nodes	2
Virtualization	Intel VT-x

Runtime Composition (56 Cores)



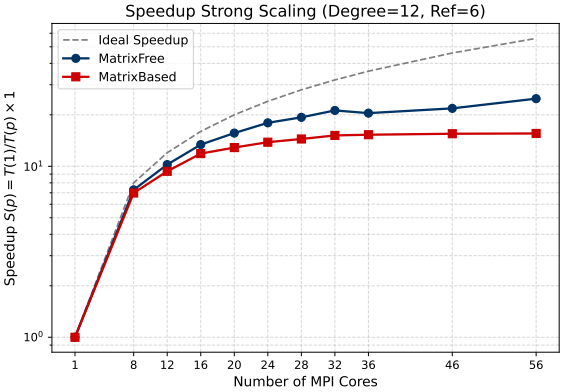
Absolute Runtime Comparison

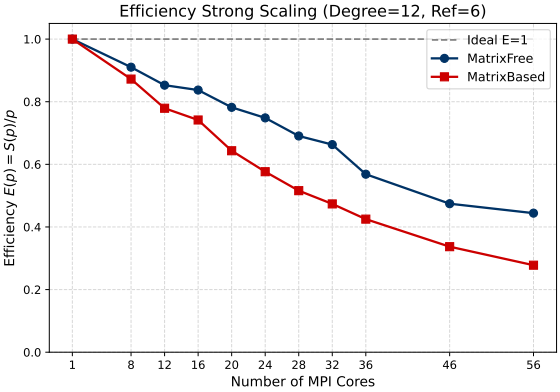


Runtime Fractional Composition

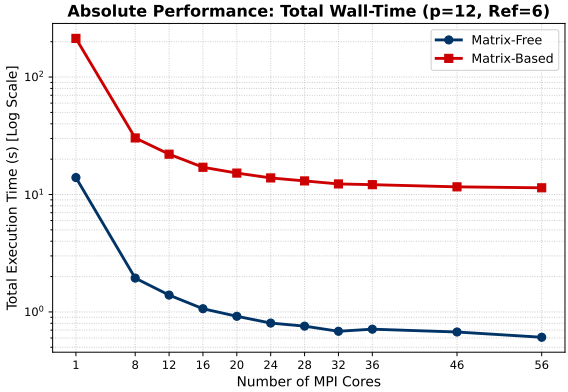
- Matrix-Based setup costs increases significantly with high polynomial degrees.
- Matrix-Free is almost entirely solve-bound, maintaining excellent scaling performance.

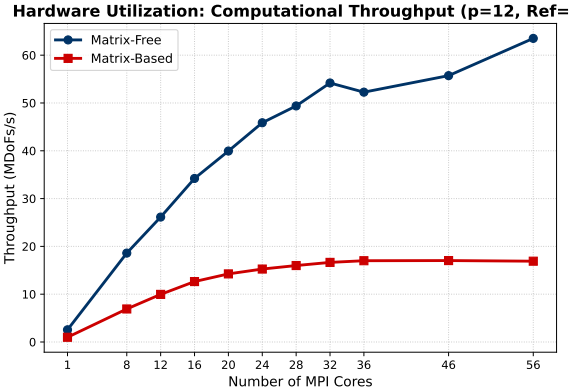
Strong Scalability



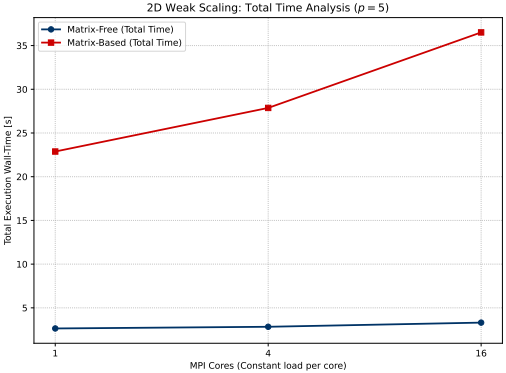


Better Metrics: Wall-Time and Throughput





Weak Scalability



Weak Scaling Wall-Time ($p = 5$, 2D)

- Scalability study limited by hardware constraints.
- Matrix-Based solver degrades as distributed sparse matrix assembly saturates memory bandwidth.